

Sistemas Operativos

2º Parcial 1C2026 TM – Resolución

Aclaración: La mayoría de las preguntas o ejercicios no tienen una única solución. Por lo tanto, si una solución particular no es similar a la expuesta aquí, no significa necesariamente que la misma sea incorrecta. Ante cualquier duda, consultar con el/la docente del curso.

Teoría

- 1- a) Segmentación sufre fragmentación **externa**. Segmentación Paginada fragmentación **interna**.
b) Es una interrupción **Sincrónica** (o excepción/trap).
c) En su entrada de **directorio**.
d) **Segmentación** (o particiones dinámicas) y asignación **Contigua** de bloques.
- 2- a) **Error** (por falta de permisos).
b) `rwxrWxr--`
c) `/home/alumnos/notas_parcial.txt`
d) `/home/alumnos/parcial.txt`, `/home/alumnos/notas1.txt` y `/home/docente/parcial.txt`
- 3-
a) Es muy probable que esté ocurriendo **thrashing**. Aumentar el grado de programación empeoraría las cosas, ya que ocasionaría **más fallos de página**.
b) A simple vista **no hay thrashing**, podemos pensar que los procesos tienen su localidad en memoria principal pero que ya hay muchos procesos ejecutando. Aumentar el grado de multiprogramación podría aumentar un poco la tasa de utilización de la CPU **siempre y cuando los procesos nuevos no generen thrashing**.
c) Podemos pensar que **hay pocos procesos ejecutando** y tienen su localidad en memoria principal. Aumentar el grado de multiprogramación aumentaría la tasa de utilización de la CPU y estamos **lejos de generar thrashing** (a diferencia del caso anterior).
- 4-
Grado de multiprogramación: En particiones fijas se ve **limitado** por la cantidad de particiones en que esté dividida la memoria física. En paginación esto no suele ser una limitante ya que el tamaño de los frames es pequeño, resultando en muchas más particiones (frames) que procesos en ejecución.
Espacio libre: En particiones fijas solamente se requiere un array con la **base y límite** de cada partición, luego cada una estará asignada a un proceso (ocupada) o no (libre). En paginación en cambio se suele utilizar un **bitarray** para saber si cada marco está libre u ocupado.
En el caso de paginación, se suele utilizar una **TLB** (caché de traducciones), esto se debe a que, a diferencia de asignación contigua donde se cargan en registros de CPU la base y límite del proceso, la tabla de páginas es muy grande para ser cargada por completo en CPU, requiriendo ser leída desde memoria en cada traducción.

5-

Hay **muchas opciones posibles**, comencemos por descartar qué esquemas no serían beneficiosos. Por un lado, un esquema enlazado puro sería muy lento a la hora de leer algunas porciones de los archivos grandes. Por otro lado, un esquema indexado puro tendría mucha fragmentación interna debido a tener muchos archivos “chicos”, esto podría contrarrestarse con un tamaño de bloque pequeño, pero causaría que no se puedan direccionar los archivos “grandes”.

Algunas opciones posibles son:

Con un tamaño de bloque de 2 KiB, utilizar **asignación contigua**: como ventaja, permite accesos directos en los archivos grandes sin incurrir en mucha fragmentación interna. La mayor desventaja de este esquema es la fragmentación externa, sin embargo, si todos los nuevos archivos ocupan un solo bloque y los archivos grandes son pre-existentes, esto no sería un problema a priori.

Otra opción es utilizar un esquema de **indexación mixta** (similar a UFS/Ext2): teniendo un FCB con al menos 1 o 2 punteros directos y 1 puntero indirecto triple en una configuración similar (bloques de 2 KiB y punteros de 32 bits) nos permitiría tener accesos directos sobre los archivos pequeños sin generar fragmentación interna y al mismo tiempo poder direccionar los archivos grandes, teniendo como desventaja algunas direcciones para leerlos y espacio ocupado en bloques de punteros.

Práctica

- 1- a) El bloque 33
b) 20 KiB (5 bloques)
c) 44 bytes
d) 1 TiB

2- a) Primero calculamos cuántos punteros entran en un bloque:

$$P \times B = \text{tamaño de bloque} / \text{tamaño de puntero} \rightarrow P \times B = 1024 / 4 = 256 \text{ punteros}$$

La cantidad máxima de bloques de datos direccionables es:

- Directos: 10
- Indirecto simple: 256
- Indirecto doble: $256 * 256 = 65536$

Total: $10 + 256 + 65536 = 65802$ bloques de datos

Como cada bloque pesa 1 KiB: TMA = 65802 KiB $\rightarrow 65802 / 1024 = 64.26$ MiB aprox.

Respuesta: el tamaño máximo de archivo es aproximadamente **64 MiB**.

b) $10500 / 1024 = 10.25$. La lectura empieza en el **bloque lógico 10**.

$544767 / 1024 = 531.99$. La lectura termina en el **bloque lógico 531**.

El rango de bloques de datos leídos es desde el bloque 10 al 531 inclusive, **522 bloques de datos**

Como la lectura empieza en el bloque 10, no se accede a los punteros directos.

El puntero indirecto simple cubre los bloques lógicos del 10 al 265. Se accede a 256 bloques de datos y **un bloque de punteros indirecto simple**.

Quedan: $522 - 256 = 266$ bloques. Esos 266 bloques están en la zona del indirecto doble. Como cada bloque indirecto simple puede referenciar 256 bloques de datos, hacen falta **2 bloques de punteros indirectos simples apuntados por el indirecto doble**.

Respuesta: se necesitan **526 accesos a bloques**, de los cuales: 522 bloques de datos, 3 bloques de punteros de indirección simple y 1 bloque de punteros de indirección doble

3- DL 32 bits, en paginación sabemos que la fragmentación interna máxima por proceso es casi una página, por lo que podríamos asumir que el **tamaño de página es 1 MiB**. Esto quiere decir que necesitamos 20 bits para direccionar el offset, dejando 12 bits para el nro de pág.

$$PA \text{ referencia a } OOB / B1233h \rightarrow \text{pág } B \rightarrow 11$$

Nos dice que hay 4 frames que se usan para procesos de usuario. Como vemos en las TP's los 4 frames ya están ocupados (los que tienen bit de P = 1).

$$La DF generada es 14 / B2233h \rightarrow \text{frame } 14 \rightarrow 20 \text{ (en decimal)}$$

Entonces vemos que la pág sustituida es la 2 de PB

A partir de esto analizamos qué algoritmos podrían haberse usado y cuáles no.

- **FIFO : No es posible.** FIFO elige el que hace más tiempo está cargado en memoria, en este caso el reemplazado había sido cargado en 100, mientras otros en 50, 90 o 95
- **LRU: Posible.** LRU elegiría al que hace más tiempo que no se referencia, es decir, el que tenga el menor TUR. En este caso el elegido había sido referenciado por última vez en 100 mientras el resto de las opciones en 105, 200 y 300.
- **CLOCK: Posible.** Tanto la víctima como PO PA tenían el bit de uso en 0. Por un lado vemos que esta última fue cargada hace más tiempo, pero también fue referenciada más recientemente. Por lo tanto, el puntero podría estar apuntando a la víctima que se eligió.
- **CLOCK MODIFICADO: No es posible.** Se elige como víctima una pág con $U = 0$ y $M = 1$ mientras que PO PA tiene $U = 0$ y $M = 0$

4-

a) - Si la dirección lógica 9FFFh corresponde al Segmento 2:

Las direcciones de 16 bits se componen en: **2 bits** para el n° segmento y **14 bits** de offset.

Entonces: 9FFFh = 10|01 1111 1111 1111

- Corresponde al **Segmento 2**, con offset 1FFFh, si es la última dirección lógica válida podemos deducir que su **tamaño es 2000h = 8 KiB**. Por otro lado, al coincidir el offset con la dirección física generada podemos deducir que el mismo se encuentra al comienzo de la memoria física (**base 0**).

- La dirección lógica 0A98h genera la dirección física 9A98h.

Entonces: 0A98h = 00|00 Corresponde al **Segmento 0** con offset A98h.

Si genera la dirección física 9A98h => **Base** = 9A98h - A98h = **9000h** (36K)

- La dirección lógica 51FAh genera la dirección física E1FAh.

Entonces: 51FAh = 01|01 Corresponde al **Segmento 1** con offset 1FAh.

Si genera la dirección física E1FAh => **Base** = E1FAh - 1FAh = **D000h** (52K)

- Sabiendo que el Segmento 1 está situado al final de la memoria, su **tamaño es 12 KiB**.

Tabla de segmentos A:

	Base	Tamaño	Las particiones asignadas nos quedan:	
0	9000h	2000h	=> [36K, 44K)	Particiones libres: [8K, 36K) => 28KiB libres
1	D000h	3000h	=> [52K, 64K)	[44K, 52K) => 8KiB libres
2	0000h	2000h	=> [0, 8K)	

b) "Se inicializa un Proceso B con un Segmento 0 de 16 KiB. Luego, crea un Segmento 1 de 8 KiB y un Segmento 2 de 12 KiB."

- El **segmento 0 se carga en la primera partición libre** (base 2000h).

Nos quedan libres: [24K, 36K) => 12KiB libres y [44K, 52K) => 8KiB libres

- El **segmento 1, por best-fit, se carga en la segunda partición libre** (base B000h).

Nos queda libre: [24K, 36K) => 12KiB libres

- El **segmento 2 se carga en la única partición libre, junto al segmento 0** (base 6000h).

Queda toda la memoria ocupada.

"El proceso A finaliza."

Esto nos **libera las particiones** que el proceso A tenía ocupados.

"El proceso B crea un segmento 3 de 16 KiB."

Ninguna de las particiones que se liberó tras finalizar el proceso A es mayor o igual a 16 KiB, sin embargo, la sumatoria de las mismas sí lo es, por lo tanto, **se debe compactar la memoria**.

Juntamos los 3 segmentos del proceso B al comienzo de la memoria, quedándonos un gran hueco libre de 28 KiB donde cargamos el Segmento 3.

Tabla de segmentos B:

	Base	Tamaño	Nos queda una única partición libre de 12 KiB al final de la Memoria	
0	2000h	4000h	** La base de los segmentos puede variar según cómo se haya realizado la compactación, en este caso solamente se movió el segmento 1 al hueco que quedó al comienzo de la Memoria.	
1	0000h	2000h		
2	6000h	3000h		
3	9000h	4000h		